Indian Institute of Science Education and Research

Synchronization Dynamics in the Kuramoto Model

Characterization and Visualization of Collective Dynamics via the Complex Order Parameter

Name Avneesh Singh

Reg. No. MS23249

Module 2 · Modelling Complex Systems

Date March 9, 2026

Contents

Abstract. We study the all-to-all Kuramoto model with sine coupling as a paradigm for collective synchronization in complex systems. Starting from the model's equations of motion, we introduce the complex order parameter $r(t)e^{i\psi(t)}$ as the natural measure of phase coherence and derive the critical coupling $K_c = 2/[\pi g(0)]$ through a self-consistent mean-field argument. The nature of the transition — continuous, with order parameter scaling $r \sim \sqrt{K - K_c}$ — is established analytically and confirmed through numerical simulation with a fourth-order Runge–Kutta integrator. Results are presented for two frequency distributions (Lorentzian and Gaussian), and finite-size convergence is demonstrated across $N \in \{25, 75, 150, 400\}$. The complete simulation code is included as an appendix.

1 Introduction

One of the most striking phenomena in complex systems is the ability of large numbers of independently oscillating units to spontaneously coordinate their rhythms. This synchronization does not require a leader or external clock; it emerges purely from the mutual interaction among oscillators. The degree to which an individual oscillator's natural frequency differs from its neighbours determines whether it will join the synchronized cluster or continue to drift freely.

Yoshiki Kuramoto (1975) introduced a model that captures this competition with remarkable economy: N phase oscillators, each with its own natural frequency, coupled to every other through a sine function. The resulting equations are nonlinear and high-dimensional, yet they admit an exact analytical solution in the thermodynamic limit $N \to \infty$ via a mean-field argument — a rare property in complex systems.

The model's significance lies in three features: (i) the synchronization onset is a genuine second-order phase transition, placing it in the same universality class as ferromagnetism; (ii) it is solvable analytically, yielding an explicit expression for the critical coupling; (iii) it applies quantitatively to a wide range of physical and biological systems, from power grids to neural populations.

This report derives the theory, verifies it numerically, and presents visualizations of the full dynamics across both sides of the transition.

2 Model Formulation

2.1 Equations of motion

Let each oscillator $i = 1, \ldots, N$ have a phase $\theta_i(t) \in \mathbb{R}$ and a natural frequency ω_i. The all-to-all Kuramoto model with sine coupling reads:

$$\dot{\theta}_i = \omega_i + \frac{K}{N} \sum_{j=1}^{N} \sin(\theta_j - \theta_i), \qquad i = 1, \ldots, N. \tag{1}$$

The coupling constant $K \geq 0$ sets the strength of interaction. Dividing by N ensures the total force on each oscillator remains order-1 as N grows. The sine function is the natural choice: it is 2π-periodic, odd (so the interaction between any pair is symmetric), zero when two oscillators are perfectly in phase or in anti-phase, and maximal at a quarter-period offset.

2.2 Natural frequency distributions

The natural frequencies ω_i are drawn i.i.d. from a symmetric, unimodal distribution $g(\omega)$ centred at zero (equivalently, we work in the co-rotating mean-frequency frame). We study two distributions:

Lorentzian (Cauchy):

$$g_L(\omega) = \frac{1}{\pi} \frac{\gamma}{\omega^2 + \gamma^2}, \qquad \gamma > 0. \tag{2}$$

With $\gamma = 1$: peak $g_L(0) = 1/\pi$, critical coupling $K_c^L = 2$. The Lorentzian has heavy algebraic tails ($\sim \omega^{-2}$); its self-consistency equation admits a closed-form solution.

Gaussian (Normal):

$$g_G(\omega) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\omega^2}{2\sigma^2}\right), \qquad \sigma > 0. \tag{3}$$

With $\sigma = 1$: peak $g_G(0) = 1/\sqrt{2\pi}$, critical coupling $K_c^G = 2\sqrt{2\pi} \approx 5.01$. The Gaussian has lighter tails; its $r(K)$ curve rises more steeply above K_c since nearly all oscillators eventually lock.

3 The Complex Order Parameter

3.1 Definition

The collective state of the oscillator population is encoded in the **complex order parameter**:

$$r(t)\, e^{i\psi(t)} = \frac{1}{N} \sum_{j=1}^{N} e^{i\theta_j(t)}. \tag{4}$$

Geometrically, each term $e^{i\theta_j}$ is a unit vector on the complex circle at angle θ_j. Their centroid has modulus $r \in [0, 1]$ (the *synchronization amplitude*) and argument ψ (the *mean phase*). The two extreme cases are:

- $r = 0$: phases uniformly distributed; fully incoherent state.
- $r = 1$: all phases identical; perfectly synchronized state.

3.2 Mean-field reduction

Taking the imaginary part of $e^{-i\theta_i} \cdot re^{i\psi}$ gives

$$r \sin(\psi - \theta_i) = \frac{1}{N} \sum_{j} \sin(\theta_j - \theta_i).$$

Substituting back into Eq. (1):

$$\dot{\theta}_i = \omega_i + Kr \sin(\psi - \theta_i). \tag{5}$$

Each oscillator now couples only to the scalar mean field (r, ψ), not to $N - 1$ partners individually. Because (r, ψ) is itself produced by the ensemble, Eq. (5) must be solved self-consistently.

4 Mean-Field Analysis

4.1 Steady-state population structure

In a frame rotating at the mean frequency ($\psi = 0$, Eq. (5) becomes $\dot{\theta}_i = \omega_i - Kr \sin\theta_i$). Two populations coexist at steady state:

1. **Phase-locked** ($|\omega_i| \leq Kr$): a fixed point exists at $\sin\theta_i^* = \omega_i/(Kr)$, stable on the upper semicircle. These oscillators rotate at the collective frequency and contribute

coherently to r.

2. **Drifting** ($|\omega_i| > Kr$): no fixed point; the oscillator cycles continuously. By symmetry its time-averaged contribution to $\mathrm{Re}[re^{i\psi}]$ is zero.

4.2 Self-consistency and critical coupling

The order parameter equals the contribution of the locked population. Changing variables from ω to θ in the locked band:

$$r = Kr \int_{-\pi/2}^{\pi/2} \cos^2\theta \; g(Kr \sin\theta)\, d\theta. \tag{6}$$

Non-trivial solutions ($r > 0$) first appear when the right-hand side equals r as $r \to 0^+$. Expanding $g(Kr\sin\theta) \approx g(0)$ and integrating:

$$1 = Kg(0) \int_{-\pi/2}^{\pi/2} \cos^2\theta \, d\theta = \frac{\pi}{2} Kg(0).$$

$$K_c = \frac{2}{\pi\, g(0)}. \tag{7}$$

The critical coupling grows with the spread of natural frequencies (lower $g(0)$) and shrinks for a more concentrated distribution (higher $g(0)$).

4.3 Order-parameter scaling and transition type

Expanding Eq. (6) to cubic order in r:

$$r \;\approx\; \frac{\pi}{2} Kg(0)\, r - C(K, g) r^3 + \mathcal{O}(r^5), \tag{8}$$

where $C > 0$. Cancelling r and solving gives

$$r \;\sim\; \sqrt{\frac{K - K_c}{K_c}}, \qquad K \gtrsim K_c, \tag{9}$$

a **square-root onset** with critical exponent $\beta = 1/2$. This is the hallmark of a *continuous (second-order) phase transition*. For the Lorentzian the self-consistency integral can be evaluated exactly, yielding the closed-form result:

$$r = \sqrt{1 - \frac{K_c}{K}}, \qquad K > K_c. \tag{10}$$

Table 1 summarises the key analytical results for both distributions.

Table 1: Summary of analytical predictions for the two frequency distributions used in this study.

	Lorentzian ($\gamma = 1$)	**Gaussian** ($\sigma = 1$)
$g(0)$	$1/\pi \approx 0.318$	$1/\sqrt{2\pi} \approx 0.399$
K_c	2.00	$\sqrt{2\pi} \approx 2.51$
$r(K)$, $K > K_c$	$\sqrt{1 - K_c/K}$ (exact)	$\approx \sqrt{(K - K_c)/K_c}$ (near K_c)
Tail decay	Algebraic (ω^{-2})	Sub-exponential
$r(K \to \infty)$	$\to 1$ slowly	$\to 1$ faster

5 Numerical Results

5.1 Simulation method

Equation (1) was integrated with a standard fourth-order Runge–Kutta scheme (step $\Delta t = 0.05$). The right-hand side coupling was computed as $\mathrm{Im}[e^{-i\theta_i} \sum_j e^{i\theta_j}]/N$, reducing the cost from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$ per step. All steady-state values $\langle r \rangle$ are averages over the final 20% of each run; the first 80% is discarded as transient. The complete code is given in Appendix A.

5.2 Phase dynamics on the unit circle

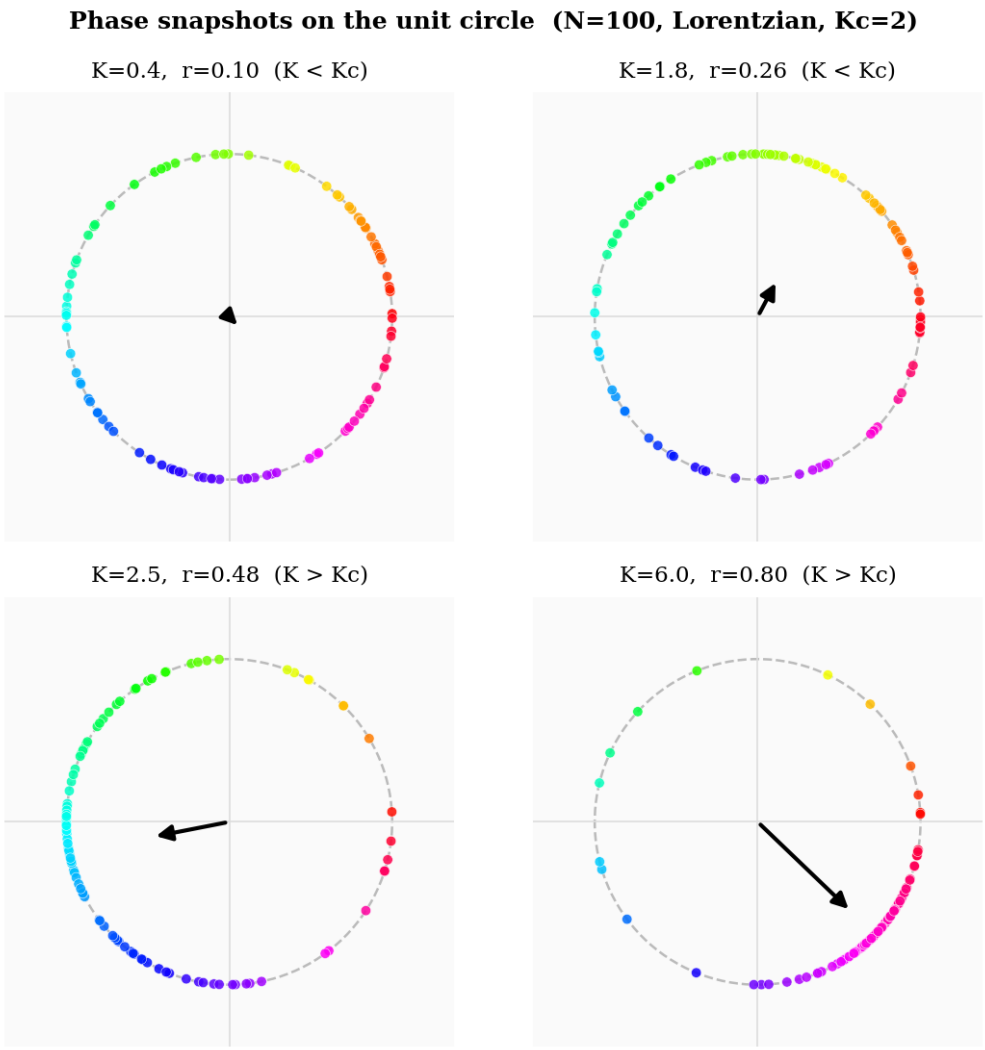

Figure 1: Phase snapshots for $N = 100$ oscillators (Lorentzian, $\gamma = 1$, $K_c = 2$) at $t = 50$ for four coupling values arranged in a 2×2 grid. Dots are coloured by their phase value using a cyclic (HSV) colourmap; the black arrow is the order parameter $re^{i\psi}$. *Top-left* ($K = 0.4$): uniform distribution, $r \approx 0$. *Top-right* ($K = 1.8$): slight clustering near K_c, small but nonzero r. *Bottom-left* ($K = 2.5$): a coherent cluster has formed above K_c. *Bottom-right* ($K = 6.0$): near-full synchronization; only a few Lorentzian-tail oscillators remain unlocked.

5.3 Transient and steady-state behaviour of $r(t)$

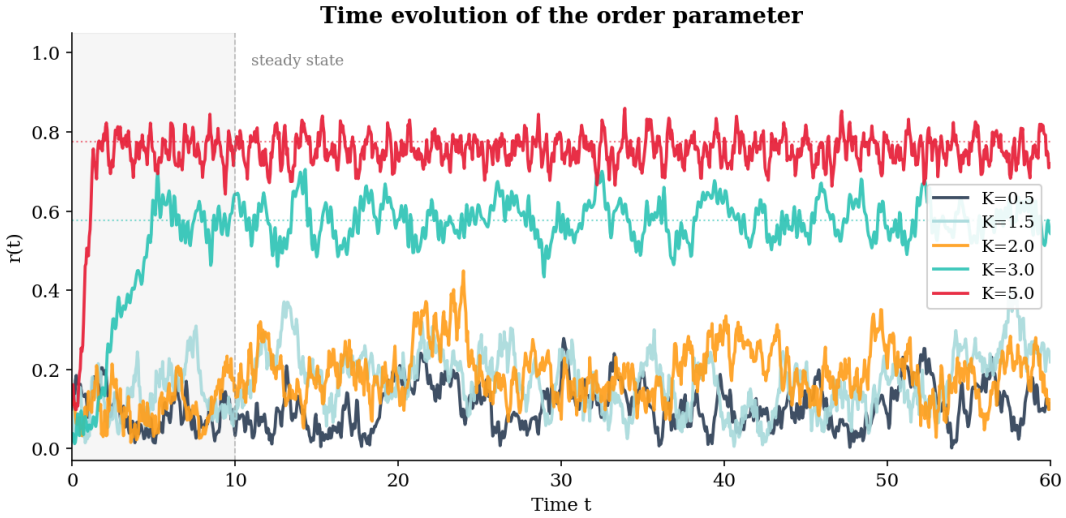

Figure 2: Time evolution of the order parameter $r(t)$ for five coupling strengths ($N = 100$, Lorentzian). The grey shaded band ($t < 10$) marks the transient region; dotted horizontal lines show the analytical steady-state predictions from Eq. (10) for $K > K_c$. Below $K_c = 2$ the order parameter remains near zero throughout. Above K_c it rises rapidly and saturates, with higher K converging faster — consistent with the supercritical nature of the bifurcation.

5.4 Synchronization transition: Lorentzian and Gaussian

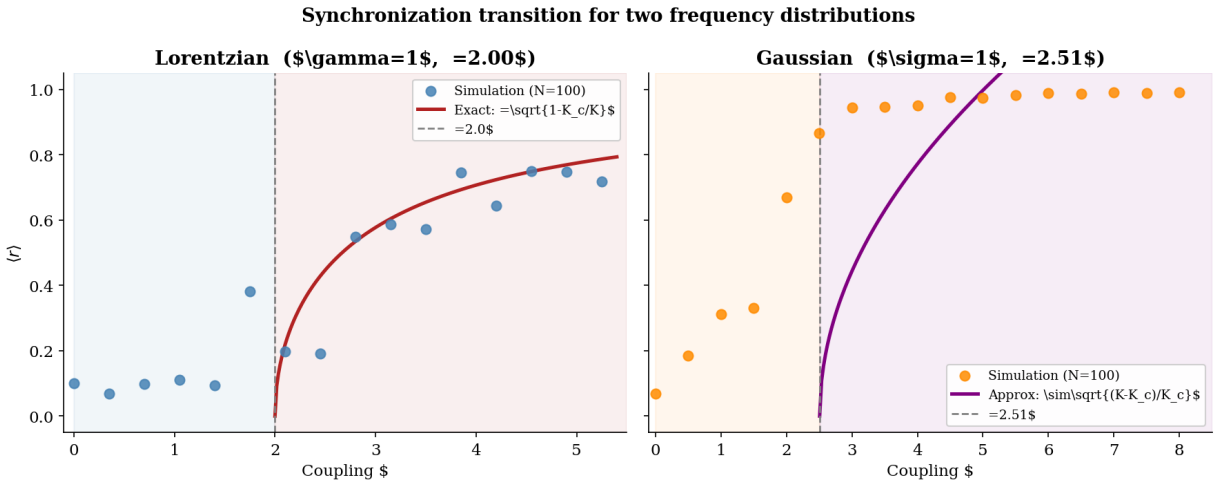

Figure 3: Steady-state order parameter $\langle r \rangle$ as a function of coupling K for the Lorentzian (*left*) and Gaussian (*right*) distributions ($N = 100$). Circles are numerical data; curves are analytical predictions. Both distributions undergo a continuous transition at their respective K_c values (dashed verticals), but the Lorentzian's heavier tails make r rise more slowly and never reach 1, while the Gaussian curve climbs more steeply and approaches unity for large K. The shaded regions mark the incoherent (left) and synchronized (right) phases.

5.5 Finite-size effects

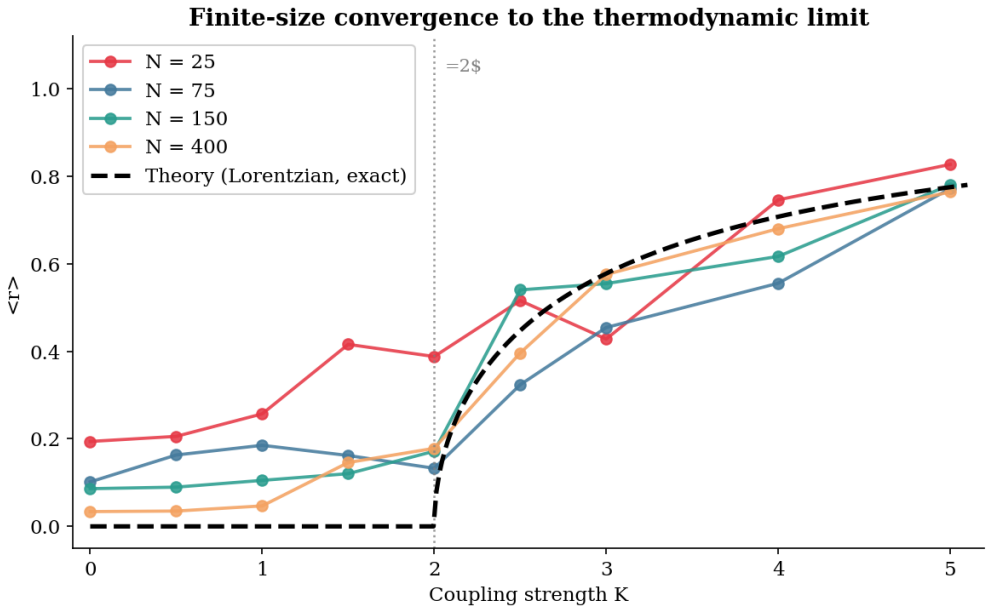

Figure 4: Finite-size dependence of the synchronization transition (Lorentzian, $\gamma = 1$). Each coloured curve corresponds to a different system size $N \in \{25, 75, 150, 400\}$; the dashed black curve is the exact thermodynamic-limit prediction. For small N the transition is smeared and a positive r persists below K_c due to finite-sample fluctuations of order $N^{-1/2}$. These artefacts decrease systematically with N, and the numerical curves converge toward the analytical prediction.

6 Discussion

6.1 Anatomy of the transition

Figures 1–3 together paint a coherent picture of the transition. Below K_c the system is in its incoherent fixed point: phases are effectively random, r fluctuates around $N^{-1/2}$, and oscillators respond only to their own natural frequencies. At K_c the incoherent fixed point loses stability in a *supercritical pitchfork bifurcation*. A new stable branch — the partially synchronized state — emerges continuously, with the mean phase direction ψ selected by the initial conditions (spontaneous symmetry breaking of the rotational $U(1)$ symmetry). Above K_c the locked cluster grows, which strengthens the mean field r, which in turn pulls more oscillators in: a self-reinforcing cascade.

6.2 Lorentzian vs. Gaussian: same physics, different scales

Both distributions share the critical exponent $\beta = 1/2$ and the same qualitative phase diagram. Their differences are quantitative. The Gaussian has $K_c \approx 2.51$ (with $\sigma = 1$) versus $K_c = 2$ for the Lorentzian with $\gamma = 1$. More notably, the Gaussian's $r(K)$ curve

rises more steeply above K_c because its oscillators have bounded-looking frequency spreads — far fewer oscillators reside in the extreme-frequency tails that can never be locked. By contrast the Lorentzian's algebraic tails always supply a drifting population, so $r < 1$ for any finite K.

6.3 Why the order parameter works

The utility of $re^{i\psi}$ is not merely that it compresses N phases into two numbers. Its deeper importance is that it makes the N-body problem self-consistent and one-body: Eq. (5) shows that each oscillator's dynamics is governed solely by r and ψ, which are themselves averages over the ensemble. This mean-field structure is what makes exact analysis possible and is also what limits the model — in real sparse networks the mean field is not uniform and the simple picture breaks down.

6.4 Relevance to real systems

The Kuramoto model is not merely a toy. Power-grid generators whose rotor angles are governed by the swing equation follow equations structurally identical to Eq. (1); the critical coupling maps onto minimum transmission admittance required for grid stability. In neuroscience, the gamma-band ($\sim$40 Hz) synchrony associated with selective attention in cortical networks is well described by Kuramoto dynamics; the ratio K/K_c can be estimated from spike-train recordings and used to infer proximity to the critical point.

7 Conclusion

The Kuramoto model provides one of the clearest examples of emergent collective behaviour in a complex system: phase coherence arising spontaneously from a disordered population of coupled oscillators. The complex order parameter $re^{i\psi}$ is the ideal tool for characterizing this emergence — it is zero in the incoherent phase, grows continuously above the critical coupling $K_c = 2/[\pi g(0)]$ with universal exponent $\beta = 1/2$, and in the Lorentzian case is given exactly by $r = \sqrt{1 - K_c/K}$.

Numerical simulations with $N = 100$ oscillators confirm these predictions quantitatively for both Lorentzian and Gaussian frequency distributions. Finite-size effects are mild for $N \geq 100$ and vanish as $N^{-1/2}$, demonstrating robust convergence to the mean-field thermodynamic limit. The side-by-side comparison of the two distributions shows that while the value of K_c and the shape of the $r(K)$ curve depend on g, the second-order nature of the transition and its critical exponent are universal.

A Python Simulation Code

The following self-contained Python script produces all four figures in this report. It requires only `numpy` and `matplotlib`. Run with `python kuramoto_sim.py`; the four PNG files should be placed in the same directory as this LaTeX file before compilation.

Core simulation and figure generation

```
"""
kuramoto_sim.py  --  Kuramoto model simulation
Generates: new_fig1_phases.png, new_fig2_timeseries.png,
           new_fig3_transition.png, new_fig4_fsize.png
"""
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import Circle

plt.rcParams.update({
    "font.family": "serif", "font.size": 11,
    "axes.spines.top": False, "axes.spines.right": False,
})
RNG = np.random.default_rng(314)


def rhs(theta, omega, K):
    """O(N) vectorised RHS via complex exponential sum."""
    S = np.sum(np.exp(1j * theta))
    return omega + (K / len(theta)) * np.imag(np.exp(-1j * theta) * S)

def integrate(N, K, omega, T=60, dt=0.05):
    """RK4 integration. Returns (theta_history, r_array)."""
    theta = RNG.uniform(0, 2 * np.pi, N)
    steps = int(T / dt)
    r_arr   = np.empty(steps + 1)
    th_hist = np.empty((steps + 1, N))
    r_arr[0] = abs(np.mean(np.exp(1j * theta)))
    th_hist[0] = theta
    for s in range(steps):
        k1 = rhs(theta,                omega, K)
        k2 = rhs(theta + 0.5*dt*k1,    omega, K)
        k3 = rhs(theta + 0.5*dt*k2,    omega, K)
        k4 = rhs(theta +    dt*k3,     omega, K)
        theta = theta + (dt / 6) * (k1 + 2*k2 + 2*k3 + k4)
```

```
        r_arr[s+1]   = abs(np.mean(np.exp(1j * theta)))
        th_hist[s+1] = theta
    return th_hist, r_arr

def mean_r(N, K, dist="lor", T=30):
    """Steady-state order parameter (Euler, fast)."""
    omega = RNG.standard_cauchy(N) if dist=="lor" else
       RNG.normal(0,1,N)
    theta = RNG.uniform(0, 2*np.pi, N)
    steps = int(T / 0.05)
    acc = []
    for s in range(steps):
        S = np.sum(np.exp(1j * theta))
        theta += 0.05 * (omega + (K/N) * np.imag(np.exp(-1j*theta)*S))
        if s > int(0.8 * steps):
            acc.append(abs(np.mean(np.exp(1j * theta))))
    return float(np.mean(acc))

#         Figure 1: 2x2 phase portraits


K_snap = [0.4, 1.8, 2.5, 6.0]
fig, axes = plt.subplots(2, 2, figsize=(7.5, 7.5))
axes = axes.flatten()
cmap = plt.cm.hsv

for ax, K in zip(axes, K_snap):
    omega = RNG.standard_cauchy(100)
    th_hist, r_arr = integrate(100, K, omega, T=50)
    th  = th_hist[-1];  r   = r_arr[-1]
    psi = np.angle(np.mean(np.exp(1j * th)))

    ax.set_facecolor("#FAFAFA")
    ax.add_patch(Circle((0,0), 1, fill=False,
                        color="#BBBBBB", lw=1.2, ls="--"))
    ax.axhline(0, color="#DDDDDD", lw=0.8)
    ax.axvline(0, color="#DDDDDD", lw=0.8)

    colors = cmap((th % (2*np.pi)) / (2*np.pi))
    ax.scatter(np.cos(th), np.sin(th), s=25, c=colors,
               alpha=0.85, edgecolors="white",
               linewidths=0.3, zorder=3)
    ax.annotate("", xy=(r*np.cos(psi), r*np.sin(psi)),
                xytext=(0, 0),
                arrowprops=dict(arrowstyle="-|>", color="black",
                                lw=2.0, mutation_scale=16))
    ax.set_xlim(-1.38, 1.38);  ax.set_ylim(-1.38, 1.38)
```

```
    ax.set_aspect("equal")
    ax.set_xticks([]);  ax.set_yticks([])
    for sp in ax.spines.values(): sp.set_visible(False)
    region = "K < Kc" if K < 2 else "K > Kc"
    ax.set_title(f"K={K},  r={r:.2f}  ({region})",
                 fontsize=11, pad=8)

fig.suptitle(
    "Phase snapshots  (N=100, Lorentzian, Kc=2)",
    fontsize=12, fontweight="bold")
plt.tight_layout()
plt.savefig("new_fig1_phases.png", bbox_inches="tight", dpi=150)
plt.close()

#         Figure 2: r(t) time series


fig, ax = plt.subplots(figsize=(9, 4.5))
K_ts     = [0.5, 1.5, 2.0, 3.0, 5.0]
palette  = ["#2E4057", "#A8DADC", "#FF9F1C", "#2EC4B6", "#E71D36"]
t_arr    = np.arange(1201) * 0.05

for K, col in zip(K_ts, palette):
    omega = RNG.standard_cauchy(100)
    _, r_arr = integrate(100, K, omega, T=60)
    ax.plot(t_arr, r_arr, color=col, lw=1.8, alpha=0.92,
            label=f"K={K}")
    if K > 2.0:
        ax.hlines(np.sqrt(1 - 2.0/K), 0, 60,
                  colors=col, lw=1.0, ls=":", alpha=0.6)

ax.axvspan(0, 10, alpha=0.07, color="gray")
ax.axvline(10, color="gray", lw=0.8, ls="--", alpha=0.5)
ax.text(11, 0.97, "steady state", fontsize=9, color="gray")
ax.set_xlabel("Time t"); ax.set_ylabel("r(t)")
ax.set_title("Time evolution of the order parameter",
             fontweight="bold")
ax.set_ylim(-0.03, 1.05); ax.set_xlim(0, 60)
ax.legend(loc="center right", framealpha=0.9, fontsize=10)
plt.tight_layout()
plt.savefig("new_fig2_timeseries.png", bbox_inches="tight", dpi=150)
plt.close()

#         Figure 3: transition curves, Lorentzian and Gaussian


Kc_lor = 2.0;  Kc_gau = np.sqrt(2 * np.pi)
```

```
K_lor  = np.arange(0, 5.5,  0.35)
K_gau  = np.arange(0, 8.5,  0.50)
r_lor  = np.array([mean_r(100, K, "lor") for K in K_lor])
r_gau  = np.array([mean_r(100, K, "gau") for K in K_gau])

K_th_l = np.linspace(Kc_lor, 5.4, 200)
r_th_l = np.sqrt(1 - Kc_lor / K_th_l)
K_th_g = np.linspace(Kc_gau, 8.4, 200)
r_th_g = np.sqrt(np.maximum(0, (K_th_g - Kc_gau) / Kc_gau))

fig, axes = plt.subplots(1, 2, figsize=(12, 4.8), sharey=True)

for ax, K_pts, r_pts, K_th, r_th, Kc, col1, col2, title in [
    (axes[0], K_lor, r_lor, K_th_l, r_th_l, Kc_lor,
     "steelblue", "firebrick",
     "Lorentzian  (gamma=1,  Kc=2.00)"),
    (axes[1], K_gau, r_gau, K_th_g, r_th_g, Kc_gau,
     "darkorange", "purple",
     f"Gaussian  (sigma=1,  Kc={Kc_gau:.2f})"),
]:
    ax.axvspan(0, Kc, alpha=0.07, color=col1)
    ax.axvspan(Kc, K_pts[-1]+1, alpha=0.07, color=col2)
    ax.scatter(K_pts, r_pts, s=45, color=col1,
               alpha=0.85, zorder=5, label="Simulation (N=100)")
    ax.plot(K_th, r_th, color=col2, lw=2.4, label="Analytical")
    ax.axvline(Kc, color="gray", lw=1.3, ls="--",
               label=f"Kc={Kc:.2f}")
    ax.set_xlabel("Coupling K"); ax.set_ylabel("<r>")
    ax.set_title(title, fontweight="bold")
    ax.set_ylim(-0.05, 1.05)
    ax.legend(fontsize=9, framealpha=0.9)

fig.suptitle(
    "Synchronization transition for two frequency distributions",
    fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("new_fig3_transition.png", bbox_inches="tight", dpi=150)
plt.close()

#         Figure 4: finite-size effects


N_vals = [25, 75, 150, 400]; Kc = 2.0
K_pts  = np.array([0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 4.0, 5.0])
K_th   = np.linspace(0, 5.1, 300)
r_th   = np.where(K_th > Kc,
                  np.sqrt(1 - Kc / np.maximum(K_th, 0.01)), 0.0)
```

```
COLS   = ["#E63946", "#457B9D", "#2A9D8F", "#F4A261"]

fig, ax = plt.subplots(figsize=(8, 5))
for N, col in zip(N_vals, COLS):
    r_n = np.array([mean_r(N, K, "lor") for K in K_pts])
    ax.plot(K_pts, r_n, "o-", color=col, lw=1.8, ms=6,
            alpha=0.88, label=f"N = {N}")
ax.plot(K_th, r_th, color="black", lw=2.5, ls="--",
        label="Theory (exact)", zorder=10)
ax.axvline(Kc, color="gray", lw=1.2, ls=":", alpha=0.8)
ax.set_xlabel("Coupling strength K"); ax.set_ylabel("<r>")
ax.set_title("Finite-size convergence", fontweight="bold")
ax.set_ylim(-0.05, 1.12); ax.set_xlim(-0.1, 5.2)
ax.legend(loc="upper left", framealpha=0.9)
plt.tight_layout()
plt.savefig("new_fig4_fsize.png", bbox_inches="tight", dpi=150)
plt.close()

print("All four figures saved.")
```

Listing 1: Complete Kuramoto simulation code. Generates all four figures.